

به نام خدا

گزارش تمرین کامپیوتری پنجم

سیگنال‌ها و سیستم‌ها-دکتر اخوان

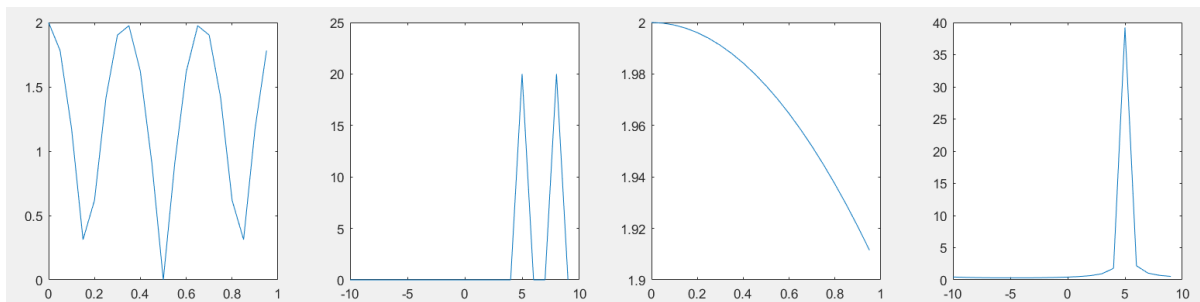
امیرمحمد میرحسینی ۸۱۰۱۰۲۶۰۱

بخش اول:

تمرین ۱-۰)

از سمت چپ در نمودار شکل اول نمایانگر سیگنال اول و شکل دوم ضرایب سری فوریه آن است. همان‌طور که مشاهده می‌شود، این ضرایب در فرکانس‌های ۵ و ۸ به بیشینه مقدار خود می‌رسند.

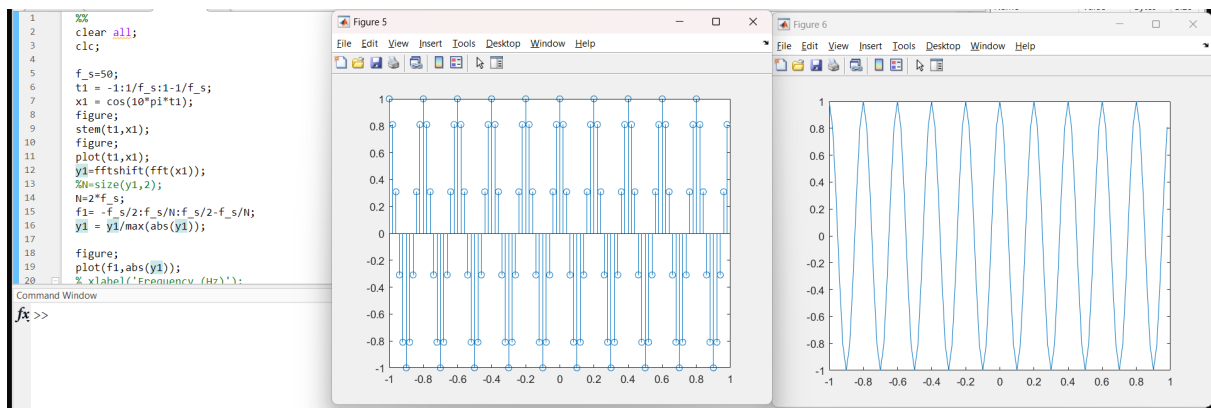
در دو نمودارهای سمت راست که مربوط به سیگنال دوم هستند، تنها در فرکانس ۵ مقدار بیشینه دیده می‌شود. در حالی که فرکانس ۵.۱ قابل تشخیص نیست، زیرا اختلاف بین این دو فرکانس کمتر از رزولوشن فرکانسی است و بنابراین تفکیک آن ممکن نبوده است.



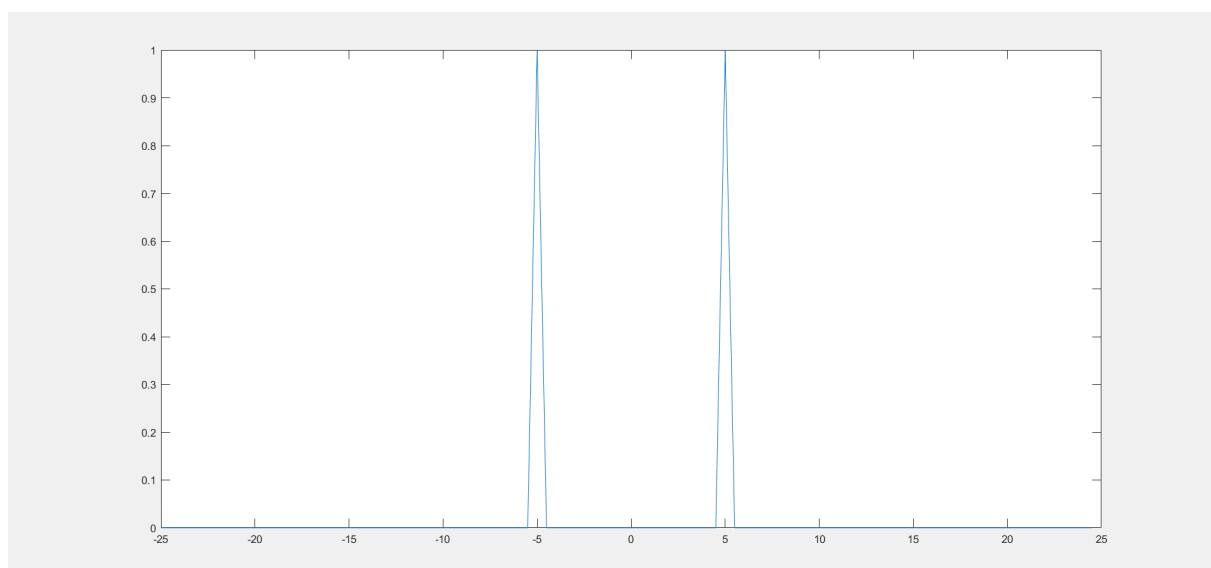
```
5 t_start = 0;
6 t_end = 1;
7 f_s = 20;
8 ts = 1/f_s;
9 N = f_s*(t_end - t_start);
10 t = t_start:ts:t_end - ts;
11 f = (-f_s/2):(f_s/N):(f_s/2 - f_s/N);
12
13 x1 = exp(1i*2*pi*5*t) + exp(1i*2*pi*8*t);
14 figure;
15 subplot(1,4,1);
16 plot(t, abs(x1));
17 y1 = fftshift(fft(x1));
18 subplot(1,4,2);
19 plot(f, abs(y1));
20
21 x2 = exp(1i*2*pi*5*t) + exp(1i*2*pi*5.1*t);
22 subplot(1,4,3);
23 plot(t, abs(x2));
24 y2 = fftshift(fft(x2));
25 subplot(1,4,4);
26 plot(f, abs(y2));
27
```

تمرین ۱-۱)

الف)



(ب)

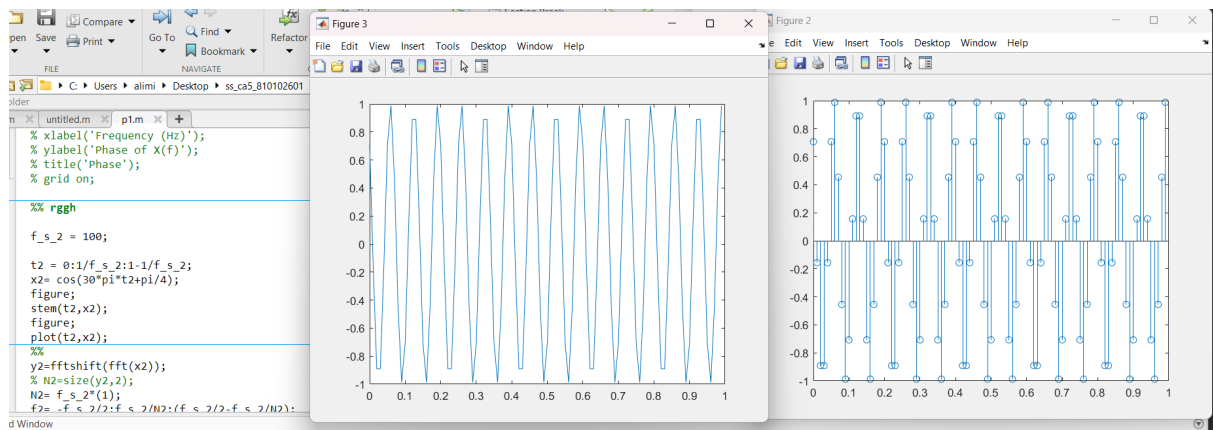


بله. این سیگنال در فضای تبدیل فوریه شامل دو مؤلفه فرکانسی ۵ و -۵ است که نمودار بالا به خوبی آن را به تصویر کشیده‌اند.

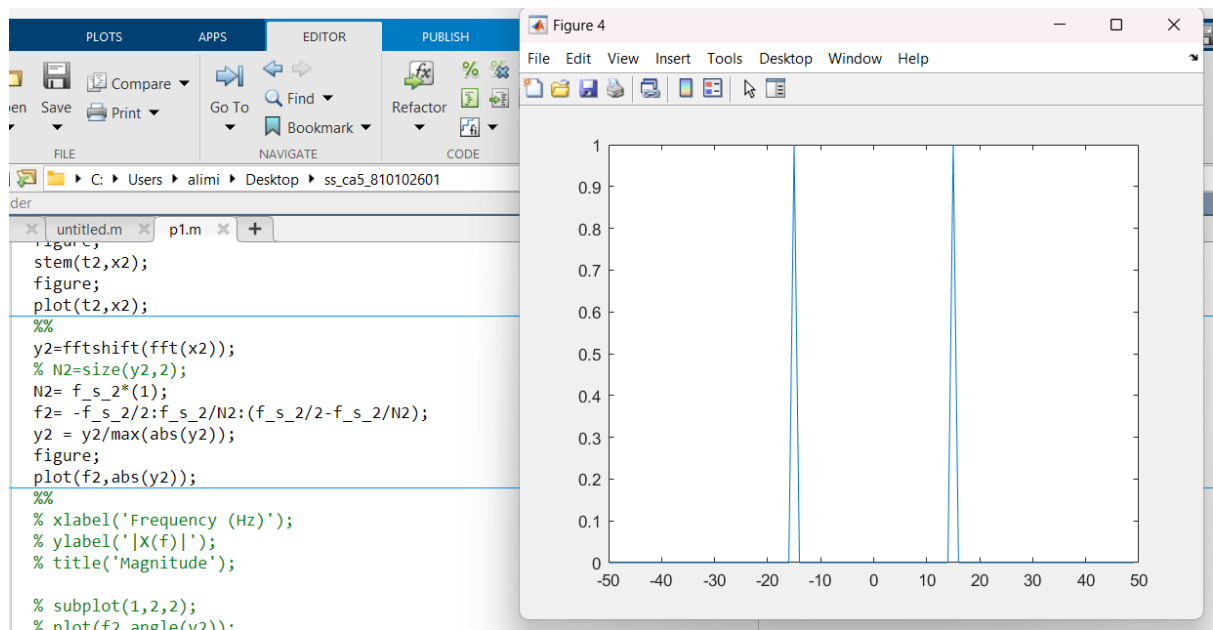
$$\cos(10\pi t) = \frac{1}{2} (e^{j10\pi t} + e^{-j10\pi t}) = \frac{1}{2} (e^{j2\pi \cdot 5t} + e^{-j2\pi \cdot 5t})$$

تمرین ۱-۲)

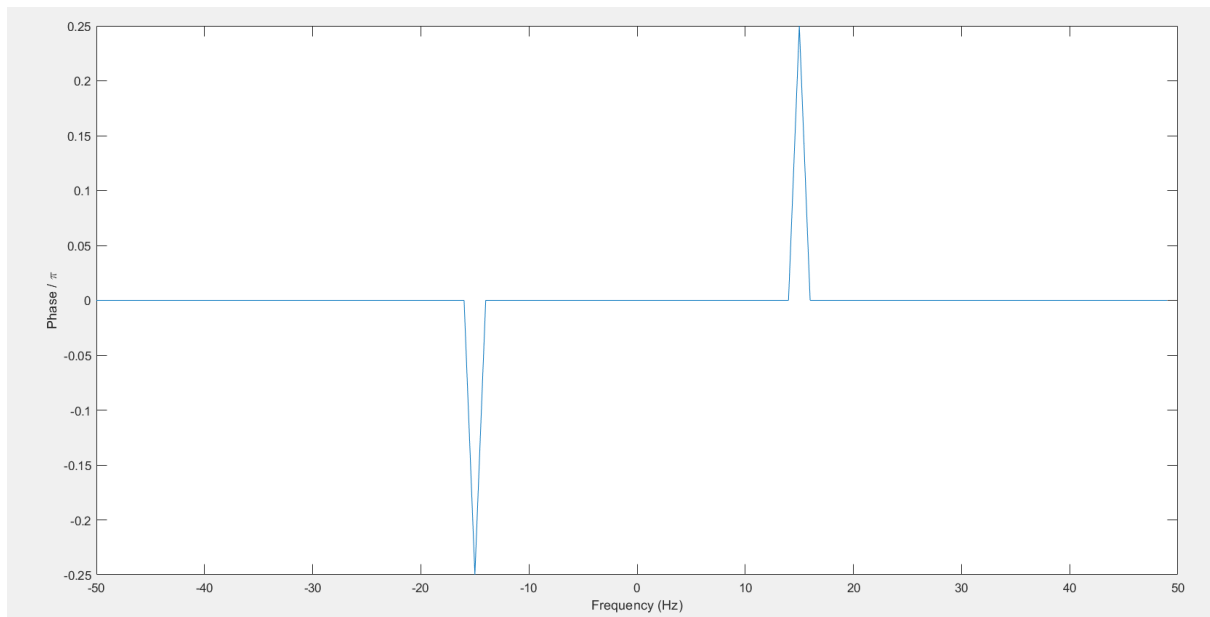
(الف)



ب) مشابه قسمت ب بخش قبل، در فوریه‌ی این سیگنال هم دو فرکانس وجود دارد ۱۵ و -۱۵ که در نمودار بالا به این نتیجه رسیده‌ایم.



ج)



با استفاده از کد ارائه شده، نمودار بالا را ترسیم کردیم. همان طور که مشخص است، سیگنال موردنظر در فضای فوریه دارای دو فاز منفی پی تقسیم بر چهار و پی تقسیم بر چهار است. بررسی نمودار نشان می دهد که نتایج به دست آمده با این مقادیر هم خوانی دارند، البته فازها بر عدد π تقسیم شده اند.

بخش دوم)

تمرین ۲-۱)

مانند تمرین های گذشته مپست را ایجاد می کنیم: تصاویر کد و خروجی مپست ضمیمه شده اند.

```
data = cell(2,32);

row1 = 'abcdefghijklmnopqrstuvwxyz ,!:"';
for i = 1:length(row1)
    data{1, i} = row1(i);
end

for i = 0:31
    data{2, i+1} = dec2bin(i, 5);
end

save('Mapset.mat', 'data');
```

Columns 1 through 11										
{ 'a' }	{ 'b' }	{ 'c' }	{ 'd' }	{ 'e' }	{ 'f' }	{ 'g' }	{ 'h' }	{ 'i' }	{ 'j' }	{ 'k' }
{ '00000' }	{ '00001' }	{ '00010' }	{ '00011' }	{ '00100' }	{ '00101' }	{ '00110' }	{ '00111' }	{ '01000' }	{ '01001' }	{ '01010' }
Columns 12 through 22										
{ 'l' }	{ 'm' }	{ 'n' }	{ 'o' }	{ 'p' }	{ 'q' }	{ 'r' }	{ 's' }	{ 't' }	{ 'u' }	{ 'v' }
{ '01011' }	{ '01100' }	{ '01101' }	{ '01110' }	{ '01111' }	{ '10000' }	{ '10001' }	{ '10010' }	{ '10011' }	{ '10100' }	{ '10101' }
Columns 23 through 32										
{ 'w' }	{ 'x' }	{ 'y' }	{ 'z' }	{ ' ' }	{ '.' }	{ ',' }	{ '!' }	{ ':' }	{ '"' }	
{ '10110' }	{ '10111' }	{ '11000' }	{ '11001' }	{ '11010' }	{ '11011' }	{ '11100' }	{ '11101' }	{ '11110' }	{ '11111' }	

```

57 %%
58 function coded = coding_freq(msg,speed)
59     load("Mapset.mat");
60     speed_freqs = [];
61     speed_step = (50/(2^speed+1));
62     for i = 1:2^speed
63         speed_freqs = [speed_freqs round(i*speed_step)];
64     end
65
66     len = length(msg);
67     binmsg = [];
68     for i = 1:len
69         disp(msg(i));
70         idx = find(strcmp(data{1,:},msg(i)));
71         kdx = data{2,idx};
72         binmsg = [binmsg data{2,idx}];
73     %     idx = find(strcmp(map{1,:}, msg(i)));
74     %     binmsg = [binmsg map{2,idx}];
75     end
76     t = 0:1/100:len/speed;
77     ts = 0:1/100:0.99;
78     y = zeros(1, length(t));
79     disp(binmsg);
80     lenb = length(binmsg);
81
82     for i=1:lenb/speed
83         s = binmsg(speed*(i-1) + 1 : speed*i);
84         jdx = bin2dec(s)+1;
85         ff = speed_freqs(jdx);
86         y((i-1)*100+1:i*100)=sin(2*pi*ff*ts);
87     end
88     coded = y;
89
90 end
91
92 function decoded= decoding_freq(signal , speed)
93

```

توضیح فانکشن: در ابتدا میست را لود می‌کنیم. سپس آرایه ای برای تولید فرکانس ها برای رمزگشایی پیام تعریف می‌کنیم. همانطور که می‌دانیم دو به توان سرعت فرکانس نیاز داریم (اعدادی که n بیت می‌سازند). اسپید استپ یا همان گام های سرعت برای تعیین فاصله بین فرکانس هاست. در حلقه بعد فرکانس ها را ایجاد می‌کنیم. از دستور راوند یا همان گرد کردن استفاده می‌کنیم تا فرکانس ها اعداد صحیح شوند.

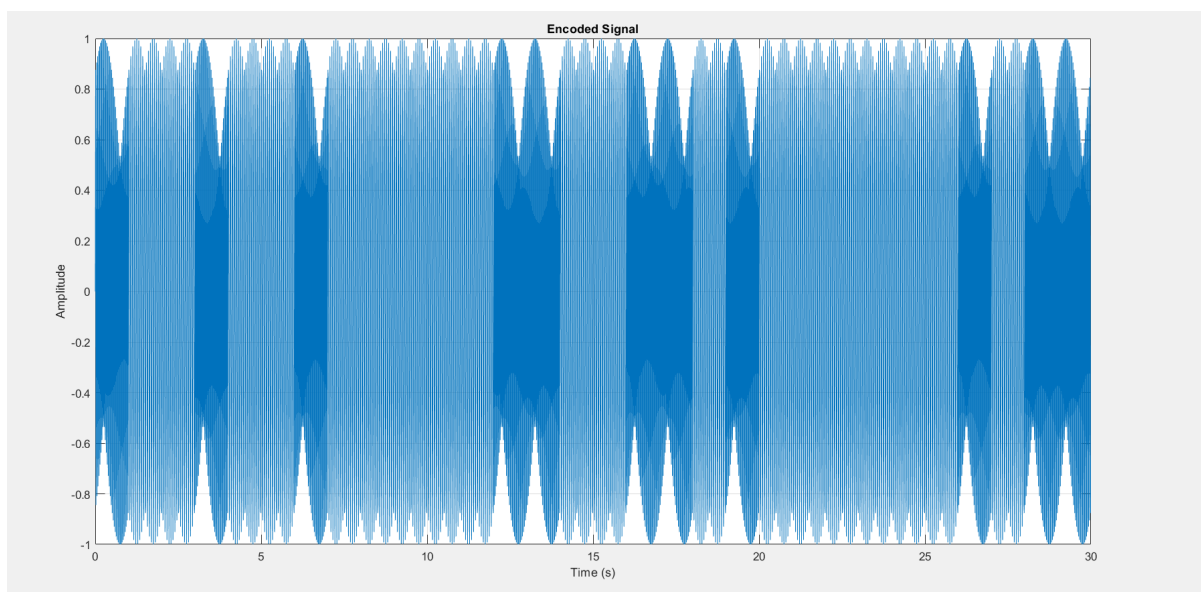
سپس یک آرایه به نام پیام باینری تولید می‌کنیم. که قصد داریم پیام باینری شده را در آن ذخیره کنیم. در حلقه بعد به ازای هر حرف موجود در رشته پیام. آن را در ردیف اول مپ ست که حروف هستند جستجو و ایندکس مربوط به آن را پیدا می‌کند. سپس مقدار آن در سطر دوم را که همان باینری شده ی حرف مورد نظر می‌باشد را ذخیره می‌کند در آرایه پیام باینری.

خب به سراغ ایجاد سیگنال کدگذاری شده می‌رویم. به اندازه سرعت مشخص شده n بیت n بیت از پیام جدا و آن قسمت جدا شده را به دسیمال تبدیل می‌کنیم و بعلاوه یک می‌کنیم چون آرایه در متلب از ۱ شروع می‌شود. و n امین عضو آرایه فرکانس های رمزگشایی پیام را انتخاب و برای یک ثانیه (که در هرثانیه ۱۰۰ نقطه یعنی یک صدم یک صدم مقدار می‌دهیم)

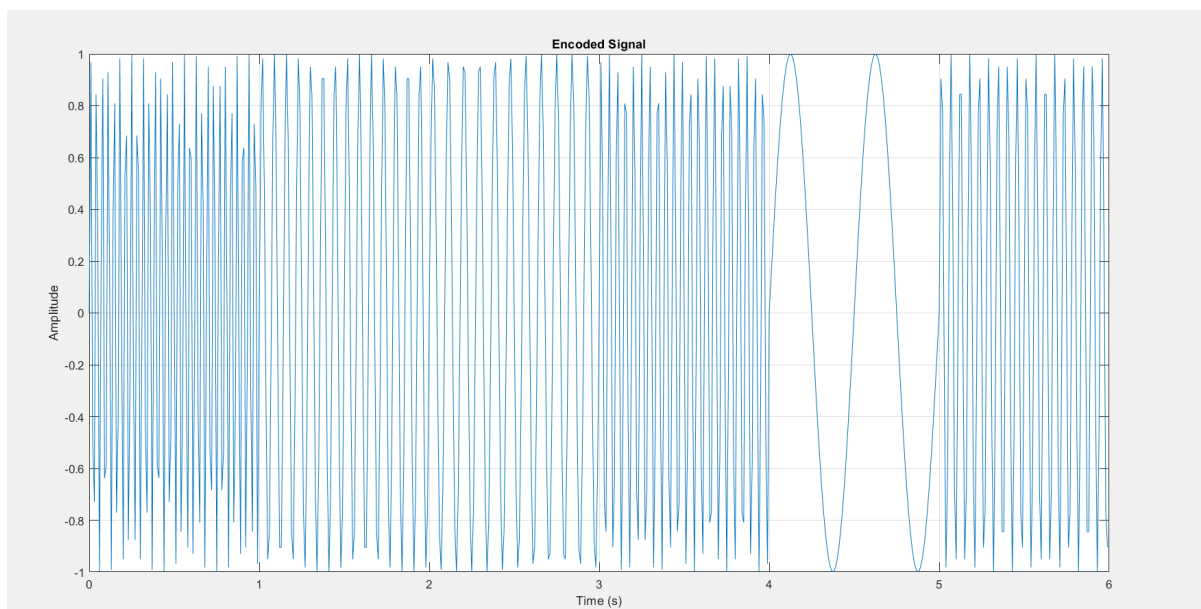
فرکانس سینوسی را ایجاد می‌کنیم. اینکار را تا جایی ادامه می‌دهیم که کل پیام را به صورت سیگنال های سینوسی با فرکانس های مختلف رمزگشایی کنیم. و سیگنال رمزگشایی شده را به خروجی می‌دهیم.

تمرین ۲-۳)

برای یک بیت بر ثانیه



برای پنج بیت بر ثانیه



```

91
92 function decoded= decoding_freq(signal , speed)
93
94     F_s = -49:1:50;
95     speed_freqs = [];
96     speed_step = (50/(2^speed+1));
97     for i = 1:2^speed
98         speed_freqs = [speed_freqs round(i*speed_step)];
99     end
100     Fourier = -speed_freqs + 51;
101     Binmsg=[];
102     len = length(signal);
103     for i = 1:len/100
104         y=fftshift(fft(signal((i-1)*100+1:(i)*100)));
105         k=max(abs(y));
106         y=y/k;
107         % disp([i , abs(y(31)), abs(y(6))]);
108
109         for j = 1:2^speed
110             if (abs(y(Fourier(j))))>0.9
111
112                 binJ = dec2bin(j-1,speed);
113                 % disp([j,binJ]);
114                 Binmsg = [Binmsg binJ];
115             end
116         end
117     end
118
119     bitcount = length(Binmsg);
120     finalmsg = [];
121     load("Mapset.mat");
122     for i =1:(bitcount/5)
123         subl = Binmsg(5*(i-1)+1:5*(i-1)+5);
124
125         jdx = find(strcmp(data(2, :), subl));
126         letter = data{1, jdx};
127         finalmsg = [finalmsg letter];
128     end
129     decoded = finalmsg;
130 end

```

توضیح فانکشن:

ابتدا آرایه فرکانس های رمزگشایی را مانند بخش کدینگ تعریف می کنیم.

سپس به دلیل استفاده از *FFTSHIFT* فرکانس ها از -۴۹ تا ۵۰ نیستند و ترتیب آنها متفاوت می شود. (طبیعتاً ترتیب در فرکانس های رمزگشایی نیز عوض می شود.) به همین دلیل ترتیب درست فرکانس ها را در آرایه جدید بنام فوریه ذخیره می کنیم که برابر است با منفی فرکانس های قبلی بعلاوه ۵۱.

یک آرایه بعنوان پیام باینری تعریف می کنیم. طول سیگنال تقسیم بر ۱۰۰ یعنی ثانیه های سیگنال و می دانیم ما سیگنال را ثانیه به ثانیه رمزگشایی کردیم بنابراین در حلقه بعد هر یک ثانیه از سیگنال را جدا کرده و به فضای فوریه می بریم. سپس با تقسیم کردن آن بر ماکسیمم اندازه Y (تبدیل فوریه شده سیگنال)، آن را نرمالایز می کنیم. سپس در یک IF تعیین می کنیم که اگر اندازه Y با اندیس به مقدار فرکانس های جدید فوریه بیشتر از ۰.۹ بود (ترشد دلبخواه) ، یعنی فرکانس

مدنظرمان پیدا شده. بنابراین اندیس فرکانس را پیدا می‌کنیم و منهای یک می‌کنیم و تبدیل به باینری می‌کنیم. باینری شده (رمزگشایی شده) آن ثانیه از سیگنال پیدا می‌شود. به همین ترتیب برای سایر ثانیه‌ها نیز باینری شده سیگنال رمزگشایی شده را پیدا و در آرایه پیام باینری ذخیره می‌کنیم.

سپس مپ ست را لود، ۵ بیت ۵ بیت از پیام باینری جدا و اندیس سطر دوم متناظر با آن در مپ ست را پیدا و سپس با اندیس ذخیره شده حرف مربوط به آن ۵ بیت باینری را پیدا می‌کنیم و در آرایه فاینال مسیج بدین ترتیب کل پیام مخفی شده سیگنال را رمزگشایی می‌کنیم.

و به خروجی تحویل می‌دهیم.

تست فانکشن:

```

5 code = coding_freq(message,1);
6 figure;
7 t = 0:1/100:(length(code)-1)/100;
8 plot(t, code);
9 xlabel('Time (s)');
10 ylabel('Amplitude');
11 title('Encoded Signal');
12 grid on;
13 decoded = decoding_freq(code,1);
14
15 figure;
16 code2 = coding_freq(message,5);
17 t = 0:1/100:(length(code2)-1)/100;
18 plot(t, code2);
19 xlabel('Time (s)');
20 ylabel('Amplitude');
21 title('Encoded Signal');
22 grid on;
23
24 decodedd = decoding_freq(code2,5);
25

```

	decoded	'signal'	12 1x6	
	decodedd	'signal'	12 1x6	

درست کار می‌کند.

تمرین ۲-۵)

```

noised_sgn1 = code + 0.01*(randn(1,length(code)));
noised_sgn2 = code2 + 0.01*(randn(1,length(code2)));
|
dcd_sgn1 = decoding_freq(noised_sgn1,1);
dcd_sgn2 = decoding_freq(noised_sgn2,5);
%%

```

dcd_sgn1	'signal'	12	1x6	d
dcd_sgn2	'signal'	12	1x6	d

بله به درستی کار می‌کند.

تمرین ۲-۶ و ۲-۷

```

35 %%
36 flag1 = 0;
37 flag2 = 0;
38 for i = 20:2:150
39
40     temp1noise = code + 0.01*i*(randn(1,length(code)));
41     temp2noise = code2 + 0.01*i*(randn(1,length(code2)));
42
43     dc1 = decoding_freq(temp1noise , 1);
44     dc2 = decoding_freq(temp2noise , 5);
45
46
47     if dc2 ~= "signal" && flag2==0
48         fprintf('decoded message for 1 bit/sec with i= %d is: %s\n: ' , i,dc1);
49         fprintf('decoded message for 5 bit/sec with i= %d is: %s\n' , i,dc2);
50         flag2=1;
51     end
52     if dc1 ~= "signal" && flag1==0
53         fprintf('decoded message for 1 bit/sec with i= %d is: %s\n: ' , i,dc1);
54         fprintf('decoded message for 5 bit/sec with i= %d is: %s\n' , i,dc2);
55         flag1=1;
56     end
57 end

```

Command Window

```

n
a
l
100100100000110011010000001011
decoded message for 1 bit/sec with i= 126 is: signal
: decoded message for 5 bit/sec with i= 126 is: signatl
decoded message for 1 bit/sec with i= 146 is: signa
: decoded message for 5 bit/sec with i= 146 is: signaly
fx >>

```

مشاهده می‌شود که همانطور که توقع داشتیم بیت ریت یک مقاومتر از بیت ریت ۵ است.

در مرتبه $i=126$ ام بیت ریت ۵ پیام دیکد شده را اشتباه دیکد می‌کند. ولی برای بیت ریت یک $i=146$ این اتفاق می‌افتد که به ترتیب برای بیت ریت ۱ و ۵ یعنی واریانس های

۵۸۷۶/۱ و ۲۱۳۱۶

تمرین ۲-۸

همانطور که در پاسخ نوشته شده، هرچه فاصله‌ی فرکانس‌های انتخابی بیشتر باشد، سیستم کدگذاری پایداری بیشتری در برابر نویز خواهد داشت. به بیان دیگر، افزایش تفکیک فرکانسی باعث بهبود مقاومت در برابر نویز می‌شود.

همچنین، مصرف پهنای باند بیشتر نه تنها امکان ارسال سریع تر اطلاعات را فراهم می کند، بلکه تاثیر نويز بر سيگنال کاهش می يابد. البته اين رويکرد نیازمند مدیریت بهينه برای منابع فرکانسی است تا بهره وری حفظ شود.

تمرین ۲-۹)

مشابه پروژه قبلی، این تغییر تأثیری نخواهد داشت، زیرا برای بهبود عملکرد باید پهنای باند را افزایش دهیم. همچنین، در مرحله کدینگ لازم است اصلاحاتی انجام شود، چرا که اگر پس از اضافه شدن نويز تغییری اعمال شود، این تغییر بر نويز نیز اثر خواهد گذاشت، اما دقت را افزایش نخواهد داد.